TreeFS — N-ary Tree in C

A data-structure-first walkthrough of node representation, core algorithms, code and Big-O complexity.

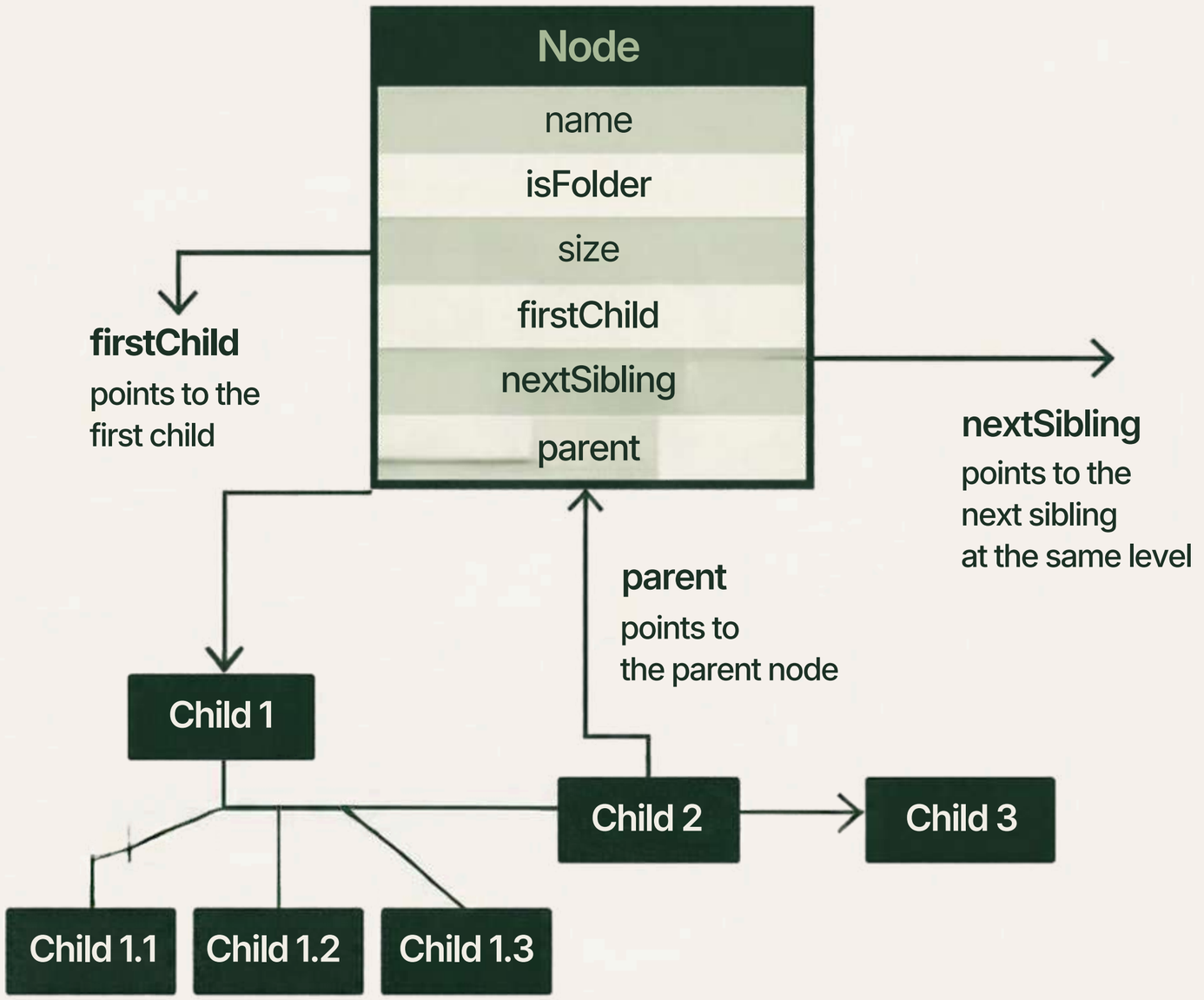


1. Node Structure & Representation

```c
typedef struct Node {
  char name[256];
  int isFolder;
  long long size;
  struct Node* firstChild;
  struct Node* nextSibling;
  struct Node* parent;
}
Node;
```

Each node stores core file information and maintains links to navigate the tree.

The **first-child / next-sibling** model represents any number of children using just two pointers. It is memory-efficient and easy to traverse.



2. createNode() + insertNode()

createNode()

```
Node* createNode(char* name,
  ichart type,
    long long size) {
  Node* n = (Node* ) malloc(sizeof(Node));
  n->name = strdup(name);
  n->type = strdup(type);
  n->size = size;
  n->firstChild = NULL;
  n->nextSibling = NULL;
  n->parent = NULL;
  return n;
```

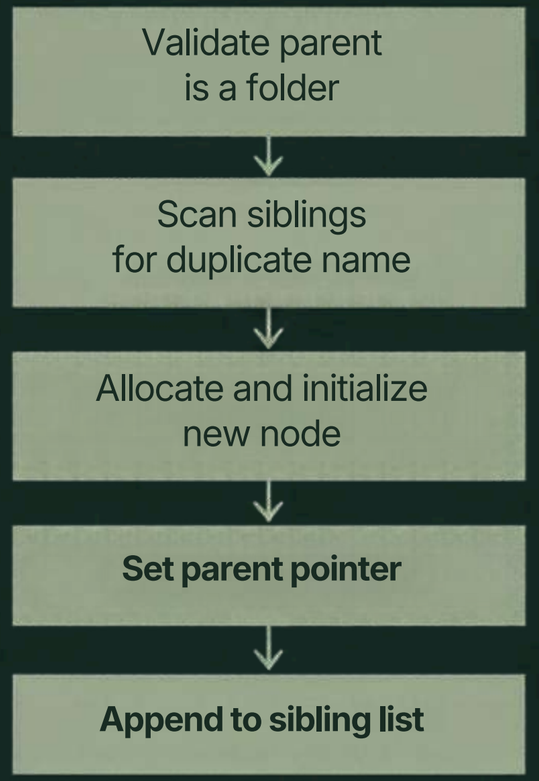
Allocates and initializes a node.
Time complexity: **O(1)**

insertNode parent, child)

```
bool insertNode(Node* parent, Node* child) {
  if (!parent || !parent->isFolder) return false;
  for (Node* s = parent->firstChild;
     s; s = s->nextSibling) {
    if (strcmp(s->name, child->name) == 0)
      return false;
}
  child->parent = parent;
  child->nextSibling = parent->firstChild;
  parent->firstChild = child;
  return true;
}
```

Validates parent, ensures no duplicate name, links child into sibling list.
Time complexity: **O(k)** where k = siblings count

INSERT WORKFLOW



PROJECT USAGE

createNode() and **insertNode()** power **buildSampleTree()** and the user **insert** command.

3. deleteNode() + renameNode()

deleteNode()

```
void deleteNode(Node* parent, Node* target) {
  if (parent |i |target) return;
  if (parent == target) return;

  prev = NULL; curr = parent->firstChild;
  while (curr &A curr != target) {
    prev = curr;
    curr = curr->nextSibling;
}
  if (curr) return;

  if (prev) prev->nextSibling = curr->nextSibling;
  else    parent->firstChild = curr->nextSibling;

  freeTree(cur);

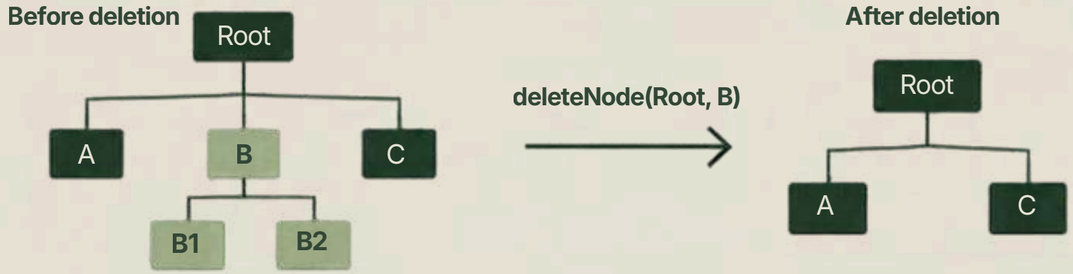
void freeTree(Node* node) {
  if (node) return;
  freeTree(node->firstChild;
  freeTree(node->nextSibling;
  free(node);
```

Root cannot be deleted. The target node is detached from its parent's child list.

Sibling links are bypassed, then the subtree is freed postorder with freeFree().

Subtree freeing is O(n); sibling detach may require scanning siblings.

Deleting a folder removes its entire contents.



renameNode()

```
bool renameNode(Node* parent, Node* target,
  const chark newName) {
  if (parent |i |target |i |newName) return false;

  curr = parent->firstChild;
  while (curr) {
    if (curr != target &A
    strcmp(curr->name, newName) == 8)
      return false;
  curr = curr->nextSibling;
}

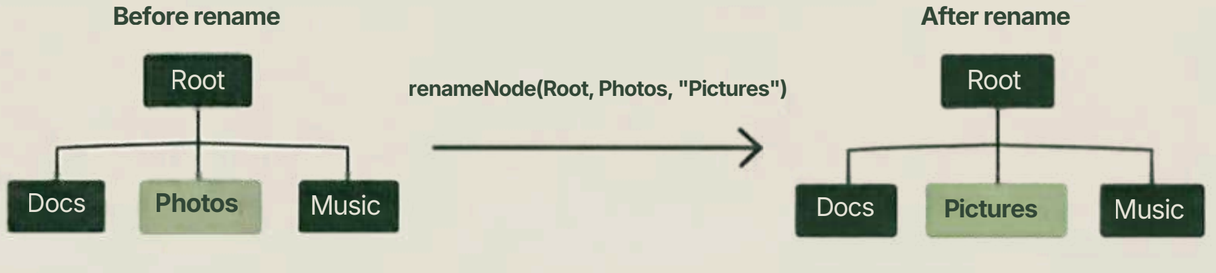
intncpy(target->name, newName, sizeof(target->name)-
>i;
target->name[si:cof(target->name)->i = ^'0';

return true;
}
```

Scans siblings to ensure the new name is unique among siblings.

If unique, the name is copied safely.

Rename runs in O(k) where i is the number of siblings.



4. moveNode() — Pointer Relinking

```
bool moveNode(const char* nodePath,
      const char* destPath) {
   Node* node = findNode(nodePath);
   Node* dest = findNode(destPath);
   if (!node || !dest || !dest->isFolder)
      return false;

   // prevent cycles
   if (isAnector(node, dest))
      return false;

   // no duplicate name in destination
   if (nameExists(dest, node->name))
      return false;

   // relink
   detachFromParent(node);
   appendChild(dest, node);
   return true;
}
```

Validate inputs

Ensure both nodes exist and destination is a folder

Prevent cycles

Use isAnecstor() to avoid moving into itself or its descendant — O(d)

Check duplicates

Ensure no same-name sibling in destination — O(k)

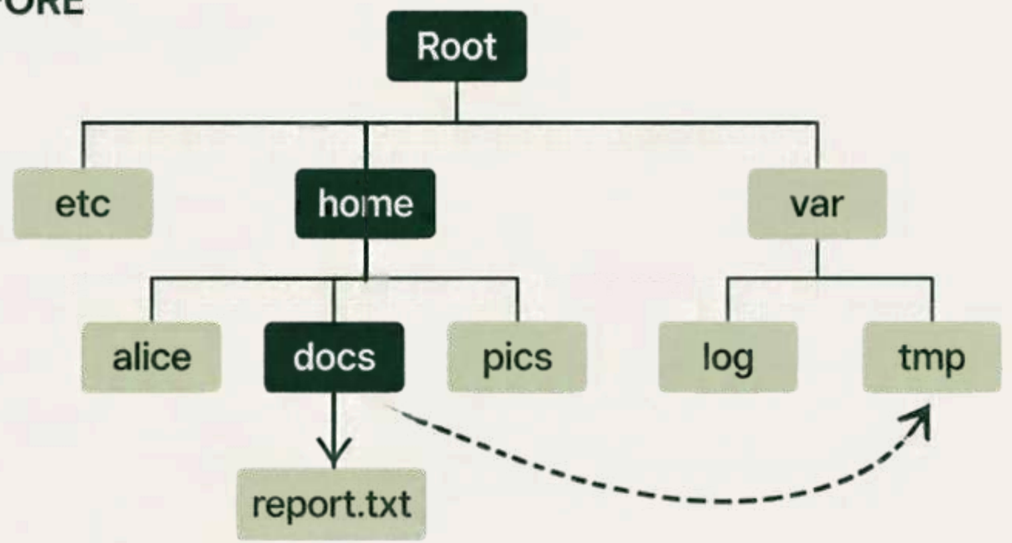
Relink pointers

Detach from old parent and append to new parent

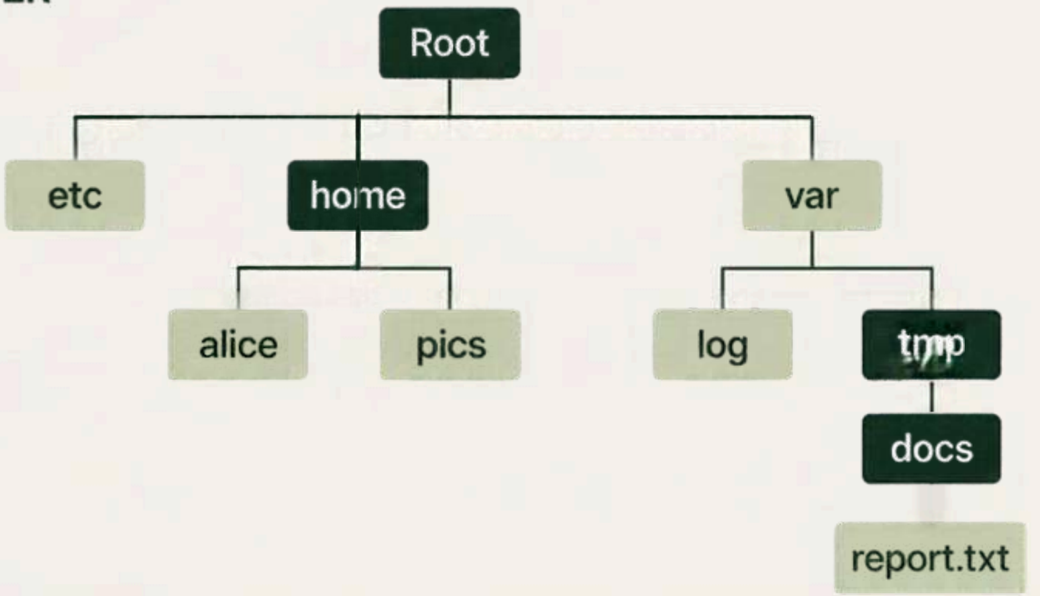
Time complexity: O(d + k)

Sibling-list scans may add additional cost

BEFORE



AFTER



5. findByPath() + searchByName()

EXAMPLE PATH

root/Projects/DS

---

findByPath tokenizes a slashseparated path and searches child lists level by level
O(dxk)

searchByName performs DFS preorder and matches substring in node names
O(n)

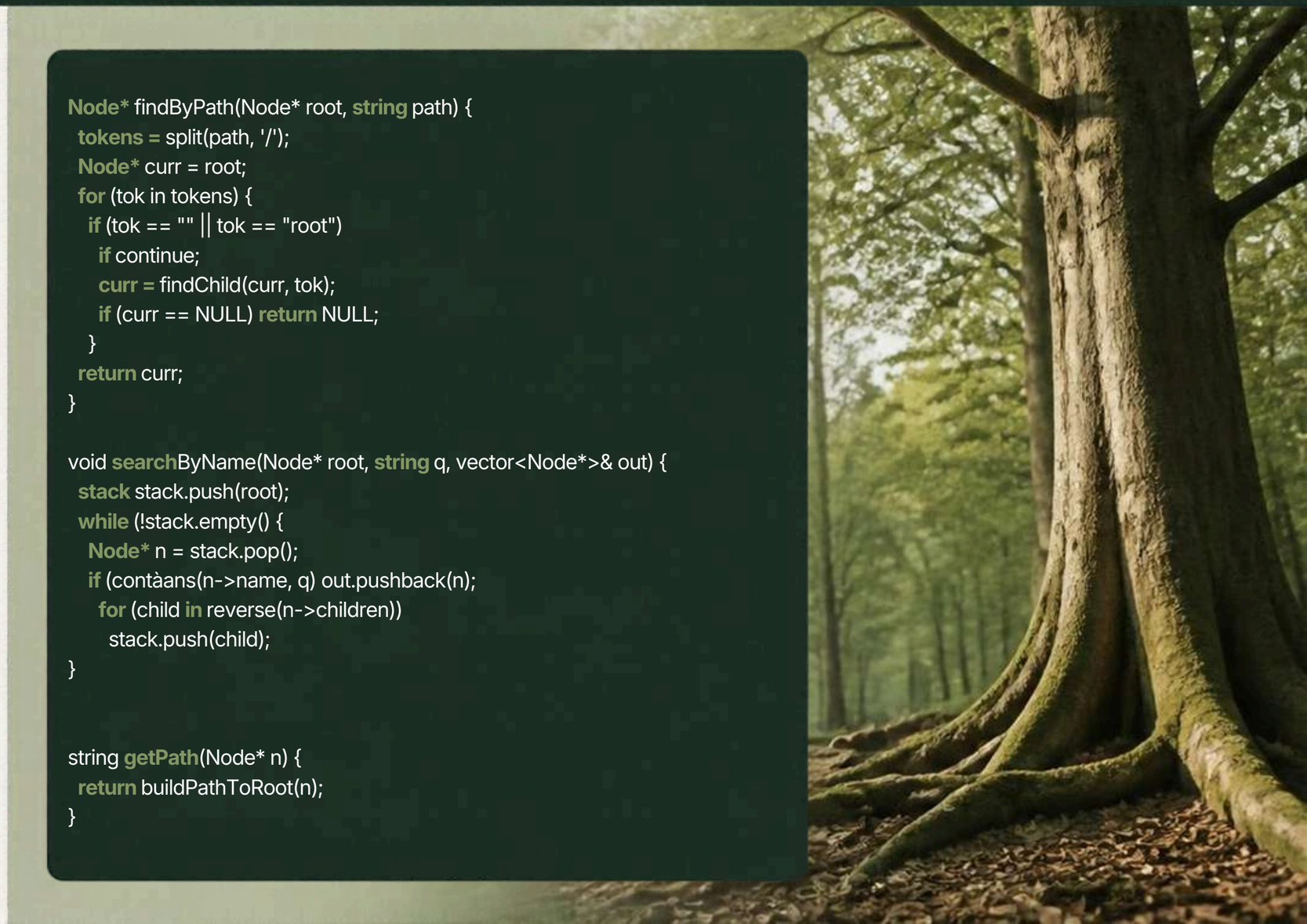
getPath builds the root-to-node path for any result

Used by insert, delete, rename, move commands and the dashboard search

```
Node* findByPath(Node* root, string path) {
  tokens = split(path, '/');
  Node* curr = root;
  for (tok in tokens) {
    if (tok == "" || tok == "root")
      if continue;
      curr = findChild(curr, tok);
      if (curr == NULL) return NULL;
    }
  return curr;
}

void searchByName(Node* root, string q, vector<Node*>& out) {
  stack stack.push(root);
  while (!stack.empty() {
    Node* n = stack.pop();
    if (contàans(n->name, q) out.pushback(n);
      for (child in reverse(n->children))
        stack.push(child);
}

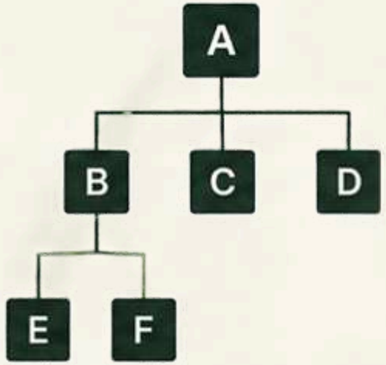

string getPath(Node* n) {
  return buildPathToRoot(n);
}
```



6. Traversals & Statistics

Explore the tree. Measure it.

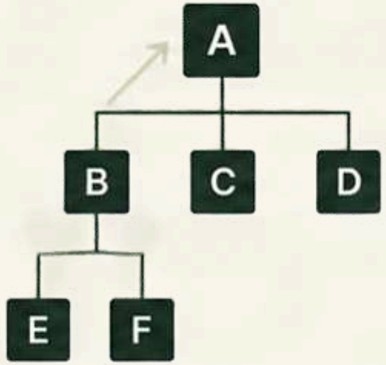
Preorder
visit parent first




```
void preorder(Node* n) {
  if (ln) return;
    visit(n);
    for (each child c in n)
        preorder(c);
}
```

Time: O(n)

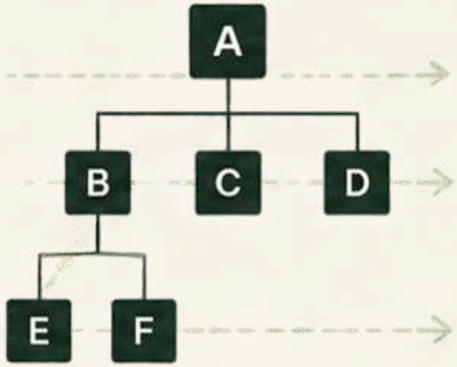
Postorder
visit children first




```
void postorder(Node* n) {
  if (ln) return;
    for (each child c in n)
        postorder(c);
    visit(n);
}
```

Time: O(n)

Level Order
breadth-first (queue)




```
void levelOrder(Node* n) {
  queue<Node> q;
  q.push(n);
  while (lq.empty() {
    Node* cur = q.pop();
    visit(cur);
    for (each child c in cur)
        q.push(c);
}
}
```

Time: O(n)

---

totalSize
recursively sums file sizes

long totalCount(Node* n);

Time **O(n)**

---

countFolders
counts folder nodes

int countFolders(Node* n);

Time **O(n)**

---

countFiles
counts file nodes

int countFiles(Node* n);

Time **O(n)**

---

treeHeight
computes height of the tree

int treeHeight(Node* n);

Time **O(n)**

7. Persistence + JSON + Project Flow

saveTree / saveNodeRec

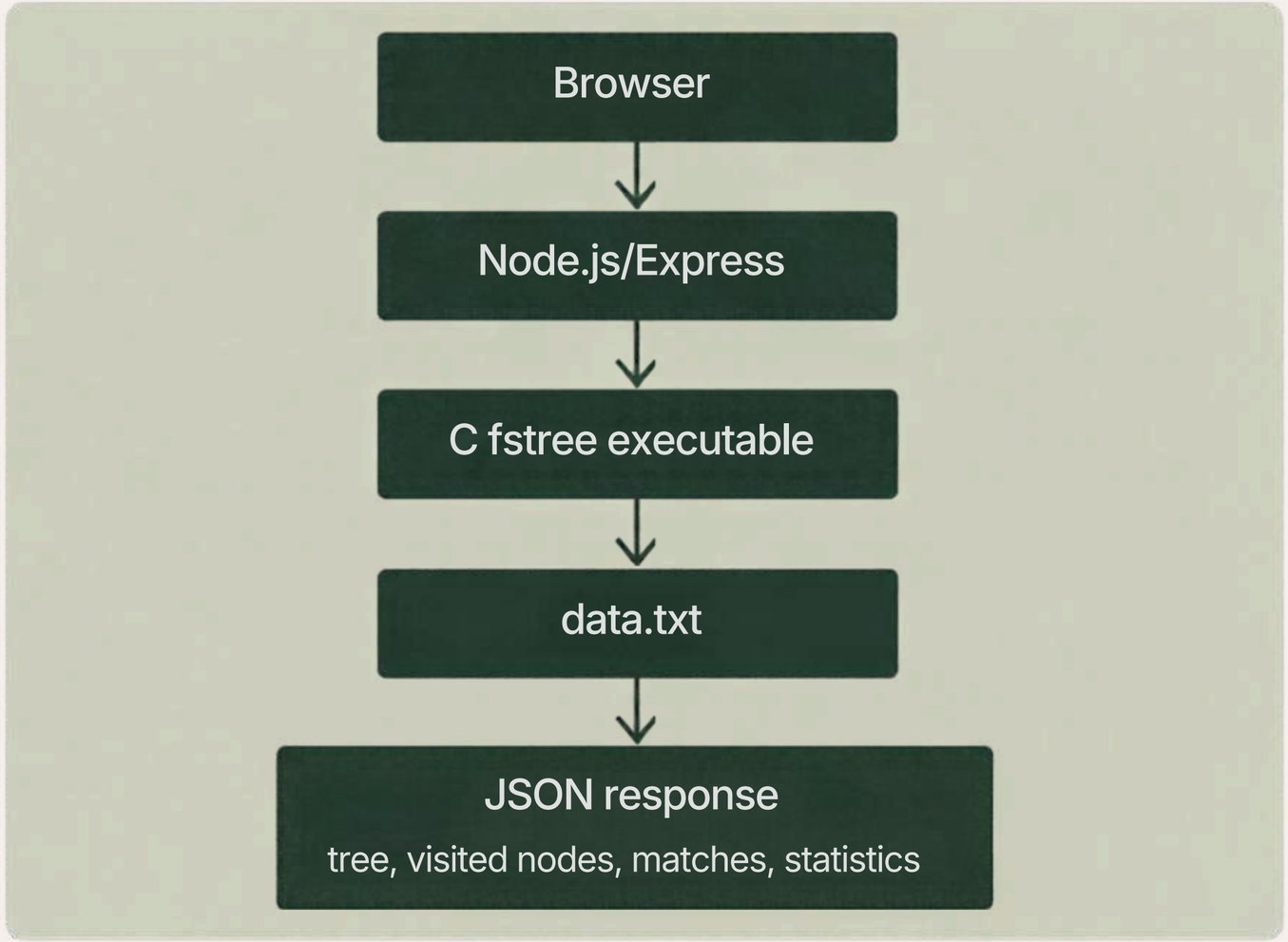
Write depth, name, type, size to data.txt in preorder traversal
Runs in O(n) time

loadTree

Read lines, use stack indexed by depth to restore parent-child links

printTreeJson / printResponse

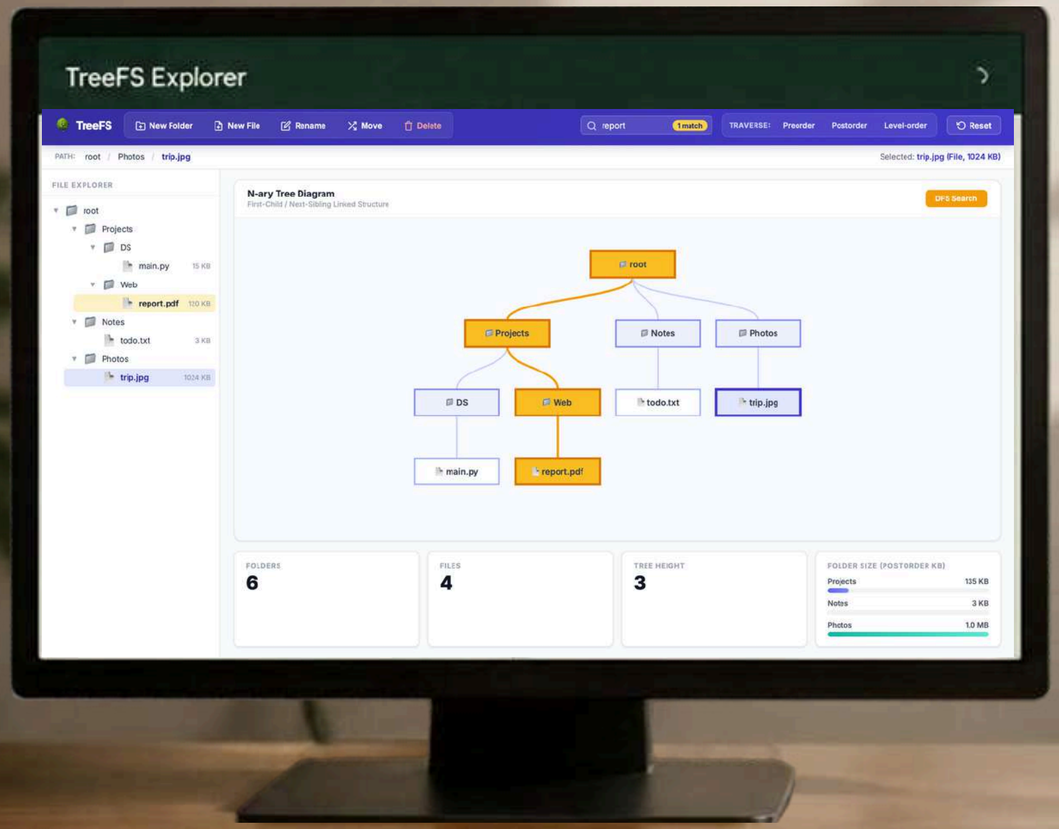
Return tree, visited nodes, matches, and statistics to the dashboard



8. Complexity Summary + Screenshot

Operation	Time Complexity
createNode	O(1)
insertNode, renameNode	O(k)
deleteNode	O(n) subtree
moveNode	O(d×k) per report
findByPath	O(d×k)
searchByName	O(n)
totalSize, countFolders, countFiles, treeHeight	O(n)
preorder, postorder, levelOrder	O(n)
saveTree, loadTree	O(n)

n = nodes visited/subtree k = children/siblings scanned d = depth



Takeaway: TreeFS operations are efficient for exploration, reporting, and persistence